

Prueba Técnica (Versión Simplificada)

Contexto

En este proyecto Next.js ya tienes montado:

- Una API básica con Route Handlers (`app/api`)
- Drizzle ORM contra PostgreSQL
- Una tabla de motos (`bikes`) y un mapa que las muestra en el frontend

Para ayudarte, puedes consultar estas referencias:

- Documentación de consultas en Drizzle ORM: <https://orm.drizzle.team/docs/data-querying>
- Quick Start de TanStack Query (React):
<https://tanstack.com/query/latest/docs/framework/react/quick-start>

No es necesario que cambies nada de la UI salvo que quieras hacerlo para probar mejor tu solución.

Objetivo Principal (Obligatorio)

Implementar **la lógica de reserva de motos** con expiración automática:

- Cada moto tiene un estado: `AVAILABLE` o `RESERVED` (los estados de viaje serán opcionales).
- Al reservar una moto, pasa a estado `RESERVED` y **queda bloqueada durante 20 segundos**.
- Después de 20 segundos, si no se ha iniciado viaje, la moto debe volver automáticamente a estado `AVAILABLE`.
- Toda la información debe persistir en base de datos (no solo en memoria).

Puedes implementar esta lógica como:

- **Servicios de backend / Route Handlers REST** en `app/api/bikes/...`, o
- Una capa de “servicio de dominio” en TypeScript que luego utilices desde los Route Handlers.

Lo importante es que se vea claramente **dónde y cómo se gestiona la reserva y su expiración**.

Modelo de Datos (guía)

Puedes comprobar el modelo de la base de datos:

```
// Enum de estado
export const bikeStatusEnum = pgEnum('bike_status', ['AVAILABLE',
'RESERVED']);

export const bikesTable = pgTable('bikes', {
  id: integer().primaryKey().generatedAlwaysAsIdentity(),
  status: bikeStatusEnum('status').notNull().default('AVAILABLE'),
  reservedUntil: timestamp('reserved_until', { withTimezone: true }),
  reservedBy: varchar('reserved_by', { length: 255 }),
  updatedAt: timestamp('updated_at', { withTimezone: true })
```

```
}).notNull().defaultNow(),  
  latitude: varchar('latitude', { length: 50 }).notNull(),  
  longitude: varchar('longitude', { length: 50 }).notNull(),  
});
```

Requisitos mínimos del modelo a tener en cuenta:

- **status**: **AVAILABLE** o **RESERVED**.
 - **reservedUntil**: **timestamp** indicando cuándo expira la reserva.
 - **reservedBy** (simple): un string que identifique al usuario (por ejemplo, un tu nombre o email).
-

Requisitos Funcionales del Servicio de Reserva

1. Reservar una moto (obligatorio)

- Endpoint sugerido: **POST /api/bikes/:id/reserve**
- Cuando se listen/recuperen las motos desde el frontend, se deben de recuperar **cada 5 segundos**.
- Request body mínimo:

```
{  
  "userId": "user123"  
}
```

- Comportamiento:
 - Verificar que la moto existe.
 - Verificar que la moto está **AVAILABLE**.
 - Si la moto estaba **RESERVED** pero la reserva ya expiró (**reservedUntil < now()**), debe considerarse **AVAILABLE** de nuevo (lazy expiration).
 - Si está disponible, actualizar:
 - **status = 'RESERVED'**
 - **reservedBy = userId**
 - **reservedUntil = now() + 20 segundos**
 - Devolver la moto actualizada en la respuesta.
- Errores recomendados (no es necesario que sean exactamente así, pero sí tener casos claros):
 - **404** si la moto no existe.
 - **409** si la moto sigue **RESERVED** y la reserva todavía no ha expirado.

2. Expiración automática (obligatorio)

- La reserva **dura 20 segundos**.
- Pasados esos 20 segundos, la moto debe volver a **AVAILABLE** y limpiarse la información de reserva.
- Puedes implementarlo de la siguiente forma:
 - **Lazy expiration**:
Cada vez que leas/consultes una moto o la lista de motos, si ves **status = RESERVED** y

`reservedUntil < fecha actual`, actualizas esa moto a:

- `status = 'AVAILABLE'`
- `reservedUntil = null`
- `reservedBy = null`

Con que implementes **lazy expiration** de forma clara es suficiente para cumplir el requisito.

Objetivos Opcionales

Si te queda tiempo o quieres ir un poco más allá, puedes implementar también:

1. Inicio de viaje (opcional)

- Endpoint sugerido: **POST** `/api/bikes/:id/start-ride`
- Reglas básicas:
 - Solo se puede iniciar viaje si la moto está **RESERVED**.
 - Solo puede iniciar viaje el mismo **userId** que reservó la moto.
 - Si la reserva ya expiró (han pasado los 20 segundos), debe fallar y la moto volver a **AVAILABLE**.
 - Estado recomendado: **IN_USE** (si decides añadirlo al enum).

2. Fin de viaje (opcional)

- Endpoint sugerido: **POST** `/api/bikes/:id/end-ride`
- Reglas básicas:
 - Solo se puede finalizar viaje si la moto está **IN_USE**.
 - Solo puede finalizar viaje el mismo **userId** que lo inició.
 - Al finalizar, la moto vuelve a **AVAILABLE** y se limpian los datos de reserva/viaje.

Estos dos endpoints son **opcionales**; puedes dejar solo el servicio de reserva funcionando.

Stack

- **Stack:** Next.js (App Router), Drizzle ORM, PostgreSQL, TypeScript.
 - Puedes usar **TanStack Query** en el frontend si quieres mostrar el resultado en el mapa, pero no es obligatorio implementarlo a fondo en esta versión simplificada.
-

Qué revisar para evaluar

- Al reservar una moto:
 - Se guarda en BD con `status = RESERVED`, `reservedBy`, `reservedUntil = now() + 20s`.
- Si se consulta una moto después de **más de 20 segundos**:
 - La moto vuelve a **AVAILABLE** automáticamente (lazy expiration).
- Si se intenta reservar una moto ya reservada y **no han pasado los 20 segundos**:
 - El backend devuelve un error apropiado (por ejemplo, 409).

Con esto es suficiente para considerar la prueba **aprobada a nivel funcional**. Los endpoints de inicio y fin de viaje solo suman puntos extra.